

SPACELOOP CI/CD IMPLEMENTATION PLAN

Automated GitHub Actions Testing, Quality Gates & Multi-Cloud Deployment Pipeline

VERSION: 1.0 (Production)
DATE: September 2026
TRACK: Open Innovation

TEAM: LOGIC LOOP
COLLEGE: GH Raisonni ISTU, Pune

Executive Summary: Production Quality & Zero-Regression Automation

SpaceLoop operates as a peer-to-peer micro-workspace marketplace with automated Rs. 100 UPI escrow, biometric Aadhaar tokenization, and dynamic AI matching. This document outlines the **GitHub Actions CI/CD Architecture** ensuring that all incoming commits and pull requests are automatically validated for code syntax, security policies, full frontend compilation, and zero-regression core functional audits before continuous deployment to Render and Vercel.

1. Core Pipeline Objectives & SLA Guarantees

The SpaceLoop CI/CD pipeline enforces four inviolable engineering pillars:

1. Deterministic Quality Gates	Every pull request must pass 100% of automated unit, authorization, IDOR, and security control tests before merging into main.
2. Strict Frontend Type Safety	Ensures the React 18 TypeScript codebase compiles cleanly with <code>tsc --noEmit</code> and Vite generates valid production assets with zero bundle drift.
3. End-to-End Functional Audit	Executes the complete 14-point functional test suite covering listing creation, AI scanning, booking lifecycle, check-in geofencing, and Rs. 100 escrow refund.
4. Automated Continuous Deployment	Triggers deployment webhooks to Render Web Service and confirms post-deployment container health and AI engine connectivity (<code>/api/health</code>).

2. GitHub Actions Pipeline Architecture (.github/workflows/ci-cd.yml)

The workflow triggers on pushes to main, pull requests against main, and manual workflow_dispatch runs. It executes three parallel and sequential jobs:

Job Identifier	Runtime / Environment	Key Execution Steps	Failure Policy
backend-ci	Ubuntu Latest Python 3.11	- Pip Cache Restore - Install requirements.txt - Unit Tests (unittest discover tests) 14-Point Functional Audit	Blocking PR / Push blocked on failure
frontend-ci	Ubuntu Latest Node.js 20.x	- NPM Cache Restore - Clean Install (npm ci) - TypeScript Typecheck (tsc --noEmit) - Vite Production Bundle (npm run build)	Blocking PR / Push blocked on failure
deploy-verify	Ubuntu Latest Triggered on main	- Runs only after CI jobs pass - Triggers Render Deploy Hook - Polls https://spaceloop.onrender.com/api/health - Verifies AI Engine status online	Alerting Notifies deployment status

3. Automated Test Verification Matrix

The automated CI suite runs 43 discrete verification checks across backend and frontend:

Test Suite Category	Test Count	Scope & Target Subsystems	Expected Result
Authentication & KYC	10 Tests	DigiLocker Aadhaar tokenization, Discom electricity CA verification, UPI penny drop, Seeker/Host role isolation, password hashing.	100% Pass (HTTP 200/401)
Authorization & RBAC	8 Tests	Role-based route protection, session hijacking mitigation, token revocation, host portal gating, authenticated context validation.	100% Pass (HTTP 403)
IDOR & Data Access	6 Tests	Insecure Direct Object Reference prevention across bookings, digital access passes, personal billing data, and user profile endpoints.	100% Pass (Zero Leakage)
Security & Anti-Spoofing	5 Tests	Rate limiting, CSP headers, XSS prevention, generic enumeration error consistency, and timing attack mitigation.	100% Pass
14-Point End-to-End Audit	14 Tests	Full functional lifecycle: Search -> AI Match -> Listing Creation -> Instant Booking -> AI Micro-Lease -> Geofenced Check-In -> Escrow Refund.	14/14 Operational

4. Continuous Deployment (CD) Workflow & Rollout Strategy

Deployment to Render and Vercel is connected directly to the GitHub repository:

Phase	Target Platform	Mechanism & Configuration	Health Verification
Render (WSGI API)	Render Cloud Web Service (Gunicorn + Python 3.11)	Auto-deploys on push to main via render.yaml blueprint. Pre-built SPA assets in public/ eliminate node dependency on server.	Automated poll of /api/health with 60s timeout.
Vercel (Edge SPA)	Vercel Global Edge CDN + Serverless Functions	Configured via vercel.json. Routes API requests to api/index.py and serves Vite SPA static assets from Edge cache.	Edge latency check and SPA fallback routing test.

5. Step-by-Step Implementation Checklist

Step	Action Item	Verification Command / Artifact	Status
1	Draft Implementation Plan & Architecture Dossier	Export to SpaceLoop_CICD_Implementation_Plan.pdf	COMPLETED
2	Create GitHub Actions CI/CD Workflow	Write .github/workflows/ci-cd.yml with parallel test jobs	READY TO EXECUTE
3	Add Pipeline Badges & Documentation to README	Update README.md with GitHub Actions build status badges	READY TO EXECUTE
4	Local Test Gate Dry Run	Execute npm run build & python test_all_features_functional.py	VERIFIED (100%)
5	Commit & Push to Remote Repositories	Push to kanishksingh-01/spaceloop & hack2ignite-practice	READY TO PUSH

Team Sign-off & Hack2Ignite 2026 Submission Readiness

This CI/CD implementation plan guarantees industrial-grade continuous integration and continuous deployment for SpaceLoop. All components, schemas, automated tests, and deployment targets adhere strictly to the Hack2Ignite 2026 Open Innovation track standards.

Architect & Engineering Team: Indrayani Mazumder, Kanishk Singh, Zara Quadri, Aarya Maurya.